

Product Requirements Document (PRD)

Retail Shelf Monitoring Using Computer Vision and Data Science

1. Problem Statement & Objective

- **Problem:** Manual shelf monitoring is time-consuming, error-prone, and slow to catch empty or misplaced products, leading to lost sales and poor customer experience.
- **Objective:** Build a computer-vision system that analyzes shelf images to detect products, count stock, flag empty/low-stock zones, identify misplaced items, and present results on a simple dashboard.

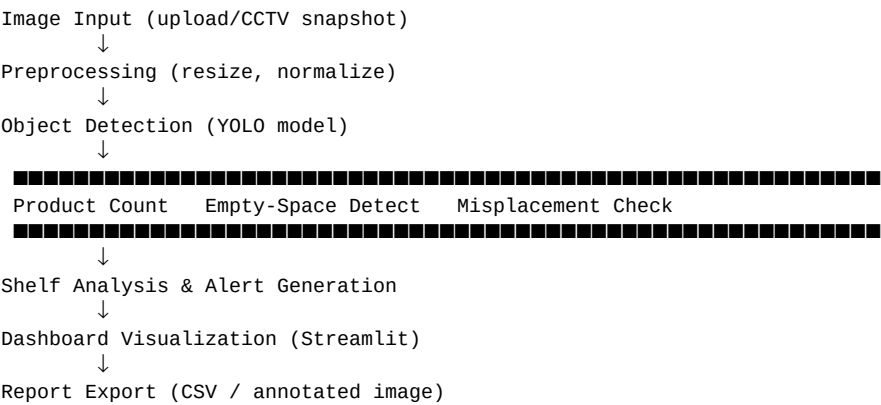
2. Target Users

- Retail store employees / shelf-stocking staff
- Store managers monitoring inventory health
- Small/medium retail chains without expensive shelf-analytics tools
- College evaluators/faculty assessing the AI/CV pipeline

3. Key Features / Functional Requirements

- Upload or capture a shelf image.
- Detect and localize products using object detection (bounding boxes).
- Count products per detected category/class.
- Detect empty/low-stock shelf segments.
- Flag misplaced products.
- Generate a shelf-health score/summary.
- Dashboard for detections, counts, and alerts.
- Downloadable CSV/annotated image report.

4. System Workflow



5. FREE/Open-Source Tools & Technologies

Purpose	Tool
Language	Python 3.10+
CV processing	OpenCV

Purpose	Tool
Object detection	Ultralytics YOLOv8
Data handling	Pandas, NumPy
Visualization	Matplotlib, Plotly
Dashboard	Streamlit
Annotation	LabelImg / Roboflow
IDE	VS Code / Jupyter / Google Colab
Version control	Git + GitHub

6. FREE/Open-Source LLM Models (Optional Layer)

Model	Size	Purpose
Phi-3-mini	3.8B	Lightweight local text summaries/alerts
Llama 3.2 3B / 1B	1B–3B	Instruction-following and report text
Gemma 2 2B	2B	Lightweight local inference

Recommendation: Llama 3.2 3B via Ollama. Optional enhancement only; not required for the core MVP.

7. FREE/Open-Source AI/ML Models & Algorithms

Task	Recommended approach
Product detection	YOLOv8n/s
Lightweight alternative	MobileNet-SSD via OpenCV DNN
Empty-space detection	OpenCV contours + rule-based gap analysis
Misplacement detection	Template/feature matching or small CNN
Product counting	Count YOLO detection outputs by class
Optional clustering	K-Means on extracted image features

Recommendation: Start with YOLOv8n because it is fast, small, and suitable for a college-scale demo.

8. Recommended Tech Stack

- **Frontend/Dashboard:** Streamlit
- **Backend:** Python; FastAPI only if a separate API layer is needed
- **Database:** SQLite
- **AI/ML:** YOLOv8n + OpenCV
- **LLM (optional):** Llama 3.2 3B via Ollama
- **APIs:** None required; optional Roboflow API
- **Deployment:** Localhost Streamlit; optionally Streamlit Community Cloud or Hugging Face Spaces

9. Dataset Requirements & Sources

- **Minimum:** 200–500 annotated shelf images with bounding boxes per product class.
- **Free sources:** SKU-110K, Grocery Store Dataset, Roboflow Universe, and self-collected shelf photos with permission.
- **Annotation:** LabelImg or Roboflow free annotation tools.

10. Evaluation Metrics

- **Detection:** mAP, Precision, Recall
- **Counting:** Mean Absolute Error (MAE)
- **Empty-shelf detection:** Accuracy/F1-score
- **Performance:** Inference time per image
- **Usability:** Dashboard clarity via peer/faculty feedback

11. MVP Scope

- Upload a single shelf image → YOLOv8n detection → bounding boxes + category counts
- Basic empty-space detection using contour/gap heuristics
- Streamlit dashboard with image, counts, and alerts
- CSV export of detection results
- **Out of scope:** real-time CCTV, multi-store dashboard, LLM summaries, and misplacement detection (stretch goals).

12. Future Enhancements

- Real-time CCTV/RTSP monitoring
- Automatic inventory system integration
- Push notifications to staff
- Product price/label OCR
- Sales prediction using historical shelf-stock data
- LLM-based daily shelf reports
- Multi-camera, multi-store historical dashboard

13. Feasibility & Estimated Resources

- **Hardware:** Standard laptop with 8GB+ RAM; Google Colab free GPU recommended for training.
- **Team:** 1–3 students; solo feasible for MVP.
- **Timeline:** Weeks 1–2 dataset; Weeks 3–5 model training; Weeks 6–7 shelf logic; Weeks 8–9 dashboard; Weeks 10–12 testing/report.
- **Cost:** ■0 using free/open-source tools and free-tier compute.
- **Risk:** Limited/imbalanced data may reduce accuracy; mitigate with augmentation and transfer learning.